Batch 3 — Situation Practice Flow

System Design Teacher Platform • Build summary with plain-language file explanations

What was delivered

Batch 1 was the foundation. Batch 2 was the front door. **Batch 3 is the first real room of the house.** A user can now log in, pick a category and difficulty, fetch a curated system-design question, write down their thoughts, reveal an AI-generated reference answer, and record that they practiced. All of this is rate-limited so a single user (or a buggy frontend) can't burn the developer's OpenAI budget.

The cost trick: AI calls happen *once*, during seeding. After that, every user gets the same pre-generated answer from the database — instant, free, served from cache. The whole platform's situation-practice mode costs roughly $1 to set up and then runs essentially for free.

Total files generated	24 (23 backend + 1 frontend zip with 47 files)
Backend lines of code (approx.)	~2,200 net new
Frontend lines of code (approx.)	~1,200 net new
New external dependencies	clsx (frontend) — 1 KB conditional class helper
New API endpoints	5 (random fetch, fetch by id, record attempt, list attempts, rate-limit status)
Frontend pages	Situation Practice (new), Home (rewired)
Curated questions seeded	50 across 10 categories × 3 difficulty levels
One-time AI cost	~$0.50 – $1.50 (gpt-4o, optional)

The big picture, in plain language

Three problems to solve, all at once.

1. **Where do questions come from?** Generating them on the fly with AI would cost money on every request and make answers vary unpredictably. Solution: *curate 50 high-quality questions ourselves, seed them into MongoDB once, and pre-generate AI answers as a one-time setup step.*

2. **How do we protect the wallet?** Even with answers cached, fetching one consumes a little compute on the server. More importantly, the same architecture will be used for the design-canvas mode in Batch 4 — where every fetch *does* cost real money. Solution: *two-layer rate limiting*: 5 fetches per user per day (fairness) AND 50 fetches across all users per day (wallet ceiling).

3. **How does the user see all this without complexity?** The frontend should make the limits visible ("4 of 5 left today"), reveal the answer only after the user has tried, let them save notes, and gracefully handle the moment they hit the limit. Solution: *a dedicated 'situation practice' page* that orchestrates filter, fetch, reveal, and record.

Two important architectural decisions

The LLM provider is now picked dynamically. The DI container has two adapter implementations of the LLM port: a `StubLLMProvider` (returns canned text, free, used in dev) and a real `OpenAILLMProvider` (calls the real API). Whichever one gets used is decided by an environment variable, with auto-detect: real API key set → use real OpenAI, placeholder key → use the stub. This means a developer cloning the repo can run everything without an OpenAI account, and the seed script is the *only* place that requires real money.

Rate limiting is two layers, not one. A per-user limit alone doesn't protect against many users at scale (200 users × 5 fetches = 1000 hits, possibly enough to exhaust budget). A global cap alone doesn't protect against one user starving everyone else. Both together compose cleanly: global is checked first (fail fast if exhausted), then per-user. ADR-004 captures this in detail.

File-by-file breakdown (plain language)

All 24 logical units, grouped by what they do.

Group 1: Documentation (1 file)

#1 `docs/adr/004-rate-limiting-strategy.md`

What it is: An Architecture Decision Record explaining the two-layer rate-limiting design, with 6 alternatives considered and explicit trade-offs noted.

Why it exists: When future-me (or a code reviewer) asks "why is this so complicated, can't we just do per-user?" — the answer is in this file. Captures the wallet-protection vs fairness distinction so the rationale doesn't get lost.

Group 2: Backend domain entities (2 files)

Pure data shapes — what the platform talks about, with no database or HTTP knowledge.

#2 `backend/app/core/domain/question.py`

What it is: Defines two Python classes: `Question` (stored in DB, has an id and timestamps) and `NewQuestion` (used during seeding, auto-fills timestamps).

Why it exists: A `reference_answer` field holds the AI-generated answer. It's nullable — questions without an answer yet show up in `list_missing_reference_answer` and become candidates for the generation script.

#3 `backend/app/core/domain/attempt.py`

What it is: Defines `Attempt` (a record that user X practiced question Y at time T) and `NewAttempt` for insertion.

Why it exists: Lets us show a user their history later. The optional `user_notes` field lets them jot what they thought before revealing the answer — capped at 5000 chars at the schema level.

Group 3: Database port modification (1 file)

#4 `backend/app/core/ports/database.py`

What it is: (Modified.) Added 6 new methods to the `QuestionRepository` contract (insert, get_by_id, random_by_filter, update_reference_answer, list_missing_reference_answer, count) and 3 to `AttemptRepository` (insert, list_by_user, count_by_user). Also formalized `ensure_indexes` on the `Database` Protocol.

Why it exists: Extending the abstract "socket" definition first locks in the contract. The MongoDB adapter must now implement these. If we ever swap MongoDB for Cosmos in Batch 5, the same methods must work — the port is the test.

Group 4: Database adapter + new OpenAI adapter (2 files)

#5 `backend/app/adapters/local/mongodb_database.py`

What it is: (Modified.) Implements the new question and attempt methods. The interesting one is `random_by_filter`, which uses Mongo's `$sample` aggregation to pick one random matching question in a single round-trip. Indexes expanded: questions(type, difficulty, category) compound for filtered fetches, attempts(user_id, created_at desc) for paginated history.

Why it exists: `$sample` is faster than count + skip + random offset because it does the work in one query. With the compound index, filtered fetches stay fast even with millions of questions.

#6 `backend/app/adapters/local/openai_llm.py`

What it is: Real OpenAI adapter. Calls `https://api.openai.com/v1/chat/completions` via `httpx`. No `openai` SDK dependency. ~150 lines.

Why it exists: The OpenAI SDK is convenient but adds a transitive dependency tree. Direct HTTP is small enough to own outright. Specific exception mapping (timeout → LLMTimeoutError, 429 → LLMRateLimitError, JSON parse failure → LLMValidationError) lets callers react meaningfully to each failure mode.

Group 5: Backend services (3 files)

Where the real business logic lives — orchestrating ports to do something useful.

#7 `backend/app/core/services/rate_limit_service.py`

What it is: ADR-004 in code form. Composes the two layers: takes an action (`SITUATION_FETCH` or `DESIGN_SUBMISSION`), checks the global counter first (fail fast if exhausted), then the per-user counter, then returns both decisions on success. Has a separate `peek` method that reads without incrementing — used by the `/rate-limits` endpoint.

Why it exists: All the rate-limit complexity in one place. Higher-up code calls `check_and_consume` and either succeeds or sees a domain `RateLimitExceeded` exception. Routes don't care about the two-layer detail.

#8 `backend/app/core/services/question_service.py`

What it is: Two methods: `fetch_random_situation_question` (consumes rate limit, fetches, logs telemetry) and `get_question_by_id` (no rate limit needed for direct lookups).

Why it exists: The fetch method enforces "every fetch counts toward the limit" by going through `RateLimitService`. There's no way to call the underlying database method without the rate-limit check from a route — by design.

#9 `backend/app/core/services/attempt_service.py`

What it is: Two methods: `record_situation_attempt` (validates the question exists, then inserts the attempt) and `list_user_attempts` (paginated, max 100 per page, sorted by recency).

Why it exists: Validating the question exists before inserting prevents orphan attempt records pointing to deleted questions. Costs an extra database read but keeps the data clean.

Group 6: API schemas (3 files)

#10 `backend/app/api/schemas/questions.py`

What it is: Pydantic models for the question API. `SituationQuestionResponse` includes the `reference_answer`; the wrapper `FetchSituationQuestionResponse` bundles the question with rate-limit metadata.

Why it exists: Putting rate-limit info in the response body (rather than HTTP headers) is simpler — the frontend doesn't need to dig into Axios's response shape.

#11 `backend/app/api/schemas/attempts.py`

What it is: `RecordSituationAttemptRequest` uses `extra="forbid"` to reject unknown fields. `PaginatedAttemptsResponse` carries the items plus total/limit/skip for pagination UI.

Why it exists: `extra="forbid"` means a buggy or malicious frontend gets a clear 422 instead of silently passing extra data.

#12 `backend/app/api/schemas/rate_limits.py`

What it is: Used only by `GET /rate-limits`. Exposes current count, limit, remaining, and reset countdown for both rate-limited actions.

Why it exists: Lets the frontend show "You have N situation fetches and M design submissions remaining today" on page load, without making the user discover the limit by hitting it.

Group 7: API routes (3 files)

Where HTTP requests come in and translate to business operations. Each route is thin — auth, parse, call service, translate, return.

#13 `backend/app/api/routes/questions.py`

What it is: Two endpoints: `GET /api/v1/questions/situation` (random with optional filters, rate-limited) and `GET /api/v1/questions/situation/{id}` (direct lookup).

Why it exists: The route has a tiny `_question_to_situation_response` helper that converts the domain entity to the API DTO. Today it's 1:1, but tomorrow if we want to hide a field or add a computed property, we change one function.

#14 `backend/app/api/routes/attempts.py`

What it is: `POST /api/v1/attempts/situation` (record one) and `GET /api/v1/attempts` (paginated history). The list endpoint accepts optional `?type=situation` filter.

Why it exists: Note the parameter naming: `type` is a Python keyword, so the param is named `attempt_type` internally with `alias="type"` on the URL. Best of both worlds — clean Python, clean URL.

#15 `backend/app/api/routes/rate_limits.py`

What it is: Single endpoint: `GET /api/v1/rate-limits`. Calls `peek()` on the rate-limit service for both actions and returns a structured snapshot.

Why it exists: Read-only and cheap (just a counter read per action), so the frontend can poll it for live feedback without affecting the user's actual budget.

Group 8: Wiring updates (4 files)

#16 `backend/app/config/settings.py`

What it is: (Modified.) Added `llm_provider` setting (literal type: 'openai' | 'stub' | None) plus an `effective_llm_provider` property that auto-detects: real key → openai, placeholder → stub.

Why it exists: Lets the app boot without an OpenAI account. New developers get the stub automatically. The stub returns canned text so endpoints work end-to-end during development.

#17 `backend/app/config/container.py`

What it is: (Modified.) Two big additions: a `providers.Selector` that picks between `OpenAILLMProvider` and `StubLLMProvider` based on settings. Three new service providers (rate-limit, question, attempt). A helper function `build_rate_limit_configs` turns scalar config values into the dict the rate-limit service expects.

Why it exists: First real `Selector` in the codebase. Same pattern Batch 5 will use to swap MongoDB → Cosmos and ConsoleTelemetry → AppInsights. Working example to copy.

#18 `backend/app/main.py`

What it is: (Modified.) Three changes: register the three new routers, add three new modules to `container.wire(modules=[...])`, and project the new settings (openai_api_key, llm_provider, rate_limits.*) into the container's config dict.

Why it exists: The `container.wire(modules=[...])` list now has 6 entries. Forgetting to add a new module here is the #1 dependency-injector footgun: `Provide[Container.x]` stops resolving silently. Hard-won lesson encoded prominently.

#19 `backend/.env.example`

What it is: (Modified.) Two updates: `MONGO_URI` changed to port 27018 (matches the Docker host-port remap from Batch 1), and a new `LLM_PROVIDER` section explaining the auto-detect behavior plus the manual overrides.

Why it exists: A developer cloning the repo can `cp .env.example .env.local` and run the seed scripts on the host without surprises. The 27018 port note prevents "connection refused" head-scratching.

Group 9: Seed scripts (4 files)

One-time setup that turns 50 hand-written question prompts into a usable dataset.

#20 `backend/scripts/__init__.py`

What it is: Empty file marking the scripts directory as a Python package.

Why it exists: Required so the scripts can do `from app.config.settings import get_settings` without import path hacks.

#21 `backend/scripts/seed_questions.json`

What it is: 50 curated situation questions, balanced across 10 categories at 3 difficulty levels (15 junior, 20 mid, 15 senior). Each entry has title, prompt (the actual question text shown to users), category, difficulty, and tags.

Why it exists: Mid-leaning because that's the level most candidates interview at. Categories cover the standard interview topics (caching, scalability, databases, security, messaging, reliability, observability, networking, data-modeling, system-tradeoffs).

#22 `backend/scripts/seed_questions.py`

What it is: Reads the JSON, finds which titles aren't already in MongoDB, inserts the missing ones. Run multiple times safely — re-running does nothing if all 50 are present.

Why it exists: Idempotent by design. Seeding is normally a once-per-environment activity, but "once" sometimes means "once per dev machine, plus once after a database reset." Idempotence makes the script forgiving.

#23 `backend/scripts/generate_answers.py`

What it is: The one-time AI script. Finds questions missing a `reference_answer`, calls `gpt-4o` for each (3 in parallel), saves the result to MongoDB. Estimated cost: $0.50 – $1.50 for all 50 questions, paid once forever.

Why it exists: Heavy safety controls: refuses to run if `OPENAI_API_KEY` is the placeholder, prompts for confirmation before spending tokens, supports `--dry-run`, `--limit N`, `--force`. Per-question token usage logged. Total cost reported at the end.

Group 10: Frontend zip (47 files in one drop-in)

The zip is a complete replacement for the `frontend/` directory. Inside: 29 files unchanged from Batch 2 (auth feature, app shell, root configs), 3 files modified, and 16 brand-new files for the situation-practice feature. Everything in one drop-in unzip.

Frontend modifications (3 files)

`src/app/router.tsx` — Added `/practice/situation` route. `src/features/home/HomePage.tsx` — Now uses the new `PageLayout`; shows two practice mode cards (Situation Practice active, Design Canvas grayed out as 'Coming soon'). `src/index.css` — Added a `.prose-content` utility class for rendering reference answers.

Frontend new shared infrastructure (6 files)

Reusable components and hooks. `Button` with primary/secondary/ghost variants, `EmptyState`, `ErrorState` (with optional retry), `Skeleton` placeholder, `PageLayout` with the nav bar, and `useRateLimitFromError` — a helper that extracts structured rate-limit info from a 429 ApiError.

Frontend situation-practice feature (12 files)

Everything for the new feature lives in one folder (*feature-sliced architecture*): TypeScript types (camelCase), Zod schemas (snake → camel transform at the API boundary), API call functions (using the shared Axios client), three TanStack Query hooks (`useSituationQuestion`, `useRecordAttempt`, `useRateLimits`), five UI components (`FilterBar`, `QuestionCard`, `AnswerReveal`, `NotesField`, `RateLimitBanner`), and the orchestrating page `SituationPracticePage`.

Notable frontend design decisions

• **Mutation, not query, for question fetch.** A `useQuery` would auto-refetch on filter change or component mount, accidentally burning rate limit. `useMutation` means "only fire when the user explicitly clicks."

• **Answer hidden behind 'Reveal'.** Pedagogical: encourages the user to think first, then check.

• **Rate limit shown three places, not redundantly:** page-level budget card (peek, no consumption), banner on the question card (from the fetch response), and a specific 429 error state when the limit is hit.

• **Snake_case → camelCase translated at the Zod boundary.** Same pattern as Batch 2's auth schemas. Components only ever see camelCase.

• **Notes capped at 5000 chars at three layers.** Browser-enforced (`maxLength` attr), JS-enforced (`slice()` in the change handler), and Pydantic-enforced on the backend.

How the pieces fit together

Here's exactly what happens when a user clicks "Fetch a question" on the situation practice page:

1. User selects filters (e.g., 'Caching' + 'Junior') and clicks "Fetch a question".
2. `useSituationQuestion.mutate(filters)` fires.
3. `fetchSituationQuestion` sends GET to `/api/v1/questions/situation?category=caching&difficulty;=junior` with the JWT in `Authorization`.
4. Vite dev proxy forwards the request to the backend on port 8000.
5. Backend's `get_random_situation_question` route handler runs.
6. `get_current_user` dependency extracts the JWT, verifies it, loads the User from Mongo.
7. Handler calls `QuestionService.fetch_random_situation_question`.
8. Service calls `RateLimitService.check_and_consume(SITUATION_FETCH, microsoft_oid)`.
9. Rate-limit service checks global key first (`global:situation_fetch:2026-04-30`) — incremented if under cap.
10. Then per-user key (`user:{oid}:situation_fetch:2026-04-30`) — incremented if under limit.
11. If either layer was at limit: raise `RateLimitExceeded` → API exception handler → 429 + structured detail. Done.
12. Otherwise, service calls `Database.questions.random_by_filter`. Mongo runs `$match` + `$sample` using the compound index.
13. Telemetry logs `situation_question_fetched` + emits `situation_question_fetch_count` metric.
14. Service returns `FetchSituationQuestionResult(question, rate_limit)`.
15. Route translates to `FetchSituationQuestionResponse` DTO and returns 200 with the JSON body.
16. Frontend's Zod schema validates the response and converts snake_case → camelCase.
17. TanStack Query stores the result. `SituationPracticePage` re-renders with the question, filter banner, and notes field.
18. User writes notes, reveals answer, clicks "Record this attempt" → second round-trip to `POST /api/v1/attempts/situation`.

Eighteen steps, but most are mechanical. The actual decisions live in steps 9–11 (rate limiting), step 12 (the random-pick aggregation), and steps 16–17 (boundary translation). Everything else is type-safe plumbing that the architecture provides for free.

How to verify Batch 3 works

Step 1 — Rebuild backend

```
docker compose up --build
```

Wait for "Uvicorn running on http://0.0.0.0:8000", "indexes_ensured", and "llm_provider: stub" in the startup log.

Step 2 — Seed the questions (instant, free)

```
cd backend
python -m scripts.seed_questions
```

Should print "Inserting 50 new questions" and a list of inserted titles. Re-running prints "All 50 questions already seeded. Nothing to do."

Step 3 — Generate answers (optional, costs ~$1)

```
python -m scripts.generate_answers --dry-run  # preview, free
python -m scripts.generate_answers --limit 1  # test on 1
python -m scripts.generate_answers           # the rest
```

Each call prompts for confirmation before spending tokens. Inspect the first generated answer in Mongo Express before running the full batch.

Step 4 — Replace frontend

```
mv frontend frontend.batch2-backup
unzip /path/to/batch-3-frontend.zip
cd frontend && npm install && npm run dev
```

Step 5 — Use the app

Navigate to `http://localhost:3000`. Sign in. Click "Situation Practice" in the nav. Pick filters, click "Fetch a question", write notes, reveal the answer, click "Record this attempt". Verify the rate-limit banner counts down. Click fetch 5 more times to see the 429 error state.

Lessons baked into this batch

• **Pre-generate, don't on-demand-generate.** AI calls are expensive at runtime and unpredictable. Doing it once during seeding turns a per-request cost into a one-time setup cost. The whole platform becomes essentially free to operate after the initial $1.

• **Two-layer rate limiting solves two distinct problems.** Per-user is fairness; global is wallet protection. Neither alone is sufficient at scale.

• **The DI `Selector` pattern is real now.** First time we have an environment-driven adapter swap (real OpenAI vs stub). Same pattern Batch 5 uses for Mongo → Cosmos.

• **Idempotent scripts are forgiving scripts.** Both seed scripts can be re-run safely. The generate-answers script even has `--dry-run`, `--limit`, and confirmation prompts. Tooling that respects the user's wallet feels good to use.

• **Use `useMutation`, not `useQuery`, for rate-limited fetches.** Auto-refetch behavior burns budget invisibly. Mutations require explicit user action.

• **Boundaries earn their keep when they get exercised.** The hexagonal architecture from Batch 1 was abstract. In this batch, the database port grew 9 methods, the LLM port got a real implementation alongside its stub, and the DI container gained a real selector. None of the existing code broke. That's the point.

What's next: Batch 4 — Design Canvas

The headline feature: drag-and-drop architecture practice with real AI feedback. The most expensive batch in cost terms (real OpenAI calls per submission) and the most ambitious frontend work.

• React Flow canvas with a draggable component palette (load balancer, cache, DB, queue, etc.)
• Diagram JSON serialization with a Zod schema
• `POST /attempts/design` endpoint that calls OpenAI for structured feedback
• `FeedbackCacheRepository` with diagram-hash-based caching (similar designs reuse cached feedback)
• Severity-grouped feedback rendering with component highlighting on the canvas
• ADR-002 (MongoDB API → Cosmos), ADR-005 (JSON over image submission)
• The rate-limit infrastructure built in Batch 3 is what protects the wallet here — every design submission consumes one of the 2-per-day per-user and 100-per-day global counters.

To begin Batch 4, say: "start Batch 4".